FreshRoute website and faculty demonstration guide

A practical handbook for using the website and explaining the complete project in clear language.

Prepared for the Cloud Architecture Design course BCSE355L. Guide date 23 October 2026. It assumes no prior knowledge of the FreshRoute interface or AWS.

What you will learn

Follow a delayed produce shipment through inspection, calculation, saved evidence, AI explanation, capacity reservation, fulfilment and reporting. Then connect each visible action to the cloud service that makes it possible. The guide includes words you can use when speaking to faculty, service-by-service explanations, and answers to common questions.

Open the website

https://dr4sw5bpoi.execute-api.ap-south-1.amazonaws.com/

Read this distinction first

The retained application is deployed in AWS Mumbai. Engineering labs were actually tested and then deleted to control cost. Some syllabus services are design studies. A study is not a deployed service. The website uses illustrative shipment data and quality assumptions, not a food-safety certification system.

This handbook was checked against the source code and recorded project evidence. It is not a claim that a new live browser demonstration was performed: the browser permission policy still blocked automated access. Screen labels below come from the implementation.

How to read this handbook

Choose the path that matches your task

Your task	Read these sections
Use the website for the first time	Sections 1 to 14 explain the screens and actions in order.
Explain the cloud architecture	Sections 15 to 30 connect services, reasons and observed evidence.
Present to faculty	Sections 31 to 33 explain architecture choices, give a timed script and answer viva questions.
Check claims and terminology	Sections 34 to 36 give limitations, glossary and references.

Three labels used throughout

RETAINED means the service is part of the deployed application or its retained operations. LAB VERIFIED AND DELETED means the service had a working experiment with recorded observations, but the temporary resources were removed. STUDY means the concept is explained through a design or comparison rather than a claimed live deployment. BLOCKED or RESTRICTED describes a specific account/access limitation.

The explanation pattern

For each website step, learn what to click, what to point out, what to say and what AWS does behind the screen. For a cloud service, learn what it is, how FreshRoute uses it, why that choice helps and what evidence supports it. This makes your explanation understandable instead of a list of AWS names.

Prepare before the evaluation

1. Open the website and decide whether you will demonstrate local mode or your own signed-in AWS workspace.
2. Prepare one fictional delayed shipment, a current saved analysis and, if available, an existing reservation/report.
3. Keep the technical report, slides and evidence files available offline. Do not reset the workspace to prepare a demonstration.
4. Rehearse the seven-minute script once. If a feature is unavailable, use its evidence and explain the limitation honestly.

1 What FreshRoute does

The problem in ordinary language

Imagine a distributor carrying tomatoes to a market. The vehicle is delayed, and the entered temperature history suggests that the produce may lose quality before arrival. Continuing is not the only possible response. Cooling or sending the produce to a different receiver may be better, but that receiver must accept the produce, have capacity and be open when the vehicle arrives.

FreshRoute organizes this decision. It records shipment assumptions, compares alternatives, explains why some are rejected, preserves the calculation and lets an operator explicitly approve a capacity-backed plan. Its purpose is decision support for dispatchers, not automatically driving a vehicle or guaranteeing a safe food donation.

The end to end story

Stage	Meaning
Observe	Inspect the shipment, delay, entered temperatures and receiving conditions.
Simulate	Calculate continuation, cooling and rerouting possibilities.
Decide	Save the evidence, review a feasible option and explicitly reserve capacity.
Verify	Follow the operational state, open the private report and inspect the cloud evidence.

What to say to faculty

> *Say to faculty: FreshRoute is a perishable shipment decision-support platform. It helps an operator compare feasible rescue options when a shipment is delayed. Its main contribution is connecting an explainable calculation with a traceable, capacity-checked decision.*

Do not claim real GPS tracking, live traffic, automated temperature sensors or actual measured food savings. Those integrations are not implemented. Current temperatures, delays, journey times and costs are entered assumptions.

2 Understand local mode and the AWS workspace

Local demo

Choose Explore the local demo on the cloud entry page, or Local demo in the workspace switch. Local mode stores fictional records in that browser and origin. It supports inspecting and editing shipments, changing receiving rules, calculating alternatives, preserving analyses and saving a local plan. A local plan is not a multi-user cloud reservation.

AWS workspace

1. Select AWS sign-in in the workspace switch, or Continue with secure sign-in on the cloud entry screen.
2. Complete sign-in on the Amazon Cognito page using your own existing account. Do not put credentials in this guide or show them to faculty.
3. After returning, check that the switch says AWS workspace. If your 15-minute session expires, sign in again.
4. Use a saved cloud analysis for the copilot, reservations, private reports and recorded cloud outcomes.

Capability	Local mode	AWS workspace
Calculate and save an analysis	Browser storage	Server recalculation and cloud persistence
AI explanation and scenario draft	Not a live model invocation	Available through the server
Shared capacity reservation	Local plan only	Conditional cloud transaction
Background import and private report	Cloud sign-in required	Cloud jobs and private storage
Outcome analytics	Local plans are not deliveries	Operator-recorded cloud fulfilment

> *Say to faculty: I am showing the AWS workspace for cloud features. Local demo mode is available for calculation practice, but I will not describe a browser-local result as a cloud transaction.*

Local data is tied to the browser, host and port. Switching from localhost to the deployed website does not transfer that browser data into the cloud.

3 Start with Overview and Shipments

What to do

1. Open Overview from the sidebar. Use it to introduce the distributor workspace and the shipment situation.
2. Open Shipments. Search for FR-101 if it exists in your current workspace. If it has been edited or removed, choose another fictional delayed shipment.
3. Use status/produce filters and the available sort controls to show how an operator finds work.
4. Open the shipment detail. Point to produce, quantity, route, entered delay, value and temperature history before opening Rescue Lab.

What the faculty should notice

The interface starts with the operational problem rather than AWS diagrams. A dispatcher first asks which shipment needs attention. Detail pages then expose the assumptions used in the comparison. A displayed quality estimate comes from the illustrative model and should not be described as a laboratory measurement.

What happens behind the screen

In cloud mode, the authenticated API loads owner-scoped shipment and workspace records from DynamoDB. In local mode, the same business journey uses the browser repository. The shared calculation model helps the local preview and authoritative server calculation follow consistent rules.

> *Say to faculty: This screen is the operator starting point. I first identify a delayed shipment, then inspect its data. The cloud API associates records with the signed-in owner, so knowing another record ID does not give me access to that user's workspace.*

Avoid a common presentation mistake

Do not immediately say that AI detected the incident. The delay and observations were entered. AI appears later, after a calculation has been saved. If you use the original worked-example numbers, verify that FR-101 still matches the unchanged seed.

4 Create and edit a shipment

Enter useful inputs

1. Choose New shipment. Enter a fictional produce type, quantity, origin and receiving destination.
2. Enter unit value, initial quality, remaining travel time, delay and expected arrival information. Keep kg, hours, degrees Celsius and rupee values consistent.
3. Add the temperature-history segments. Each segment describes a duration and temperature that the model will convert into equivalent exposure.
4. Review validation messages, then save. Reopen the shipment to confirm the saved data. Use Edit only when you intentionally want to change the source assumptions.

Why the fields matter

Quantity determines the mass being considered. Unit value supports a modeled financial comparison. Initial quality and temperature history influence the model's current quality. Future delay and temperature affect modeled arrival quality. Destination rules determine whether arrival, produce and capacity are acceptable. Bad input can produce an internally consistent but practically misleading answer.

Drafts and source versions

The local form supports preserving an unfinished draft and restoring it when reopened. Saved records have versions. When a shipment or receiving rule changes, an old analysis remains available as historical evidence, but confirmation checks whether its source is still current. This prevents an operator silently acting on obsolete assumptions.

> *Say to faculty: These fields are the evidence for the decision. I use fictional data and explicit units. Editing a source record creates a newer version; it does not rewrite a previously saved analysis.*

Suggested rehearsal record

Use a clearly named fictional shipment or the existing FR-101 seed. Do not overwrite a genuine user record simply to make the demonstration look consistent. If you create a new record during evaluation, explain that it is a new hypothetical case.

5 Use Rescue Lab to compare alternatives

The exact click path

1. Open the chosen shipment, then Rescue Lab. Start by reading the saved shipment assumptions.
2. Change one future assumption, such as additional delay or future temperature. Change the budget or ranking priority only if you can explain the reason.
3. Inspect continuation, cooling and alternative receivers. Read at least one rejection reason before discussing the top-ranked feasible option.
4. Show the side-by-side trade-off comparison and quality curves. Explain that the routes use declared assumptions, not a live maps/traffic service.
5. If there is no feasible option, explain the constraint. Do not increase capacity or budget merely to force a successful-looking result.

How to read the output

Modeled quality is a number from 0 to 100. Estimated loss is the quantity of modeled quality cohorts below the declared threshold. Cost is the declared extra cost for that option. Arrival time reflects entered journey and delay assumptions. Feasible means the option passes the implemented constraints; it does not guarantee that a real recipient will accept the food.

Constraints before preferences

Receiving hours, accepted produce, available capacity, budget and the declared quality threshold are hard checks. Only feasible candidates receive a usable rank. The weighted score combines normalized estimated loss, cost and time. A lower score is preferred among feasible options; a small score cannot overrule a failed hard constraint.

> *Say to faculty: I am not simply choosing the cheapest receiver. The application first removes invalid options and then compares the trade-offs between the valid ones. That is why a nearby or cheap receiver can still be rejected.*

The editable view is a calculated preview. In AWS mode, Run cloud analysis sends a job for authoritative server validation and calculation. The preview is not itself a saved cloud result.

6 Explain the calculation without confusing the audience

First give the intuition

Produce exposed to higher entered temperatures accumulates effective age faster in this teaching model. The app converts each history segment to equivalent time at a 10 C reference using Q10 = 2. For example, the formula assigns twice as much exposure per hour at 20 C as at 10 C. This is a modeling assumption, not a measured statement about every produce item.

Then give the formula if asked

Equivalent hours = sum of durationHours multiplied by 2 raised to ((temperatureC minus 10) divided by 10). The produce-specific exponential decay parameter then reduces each modeled quality cohort. Cohorts represent different starting quality within one batch. Their weights sum to one, and their masses sum to the shipment quantity.

Average quality and estimated loss answer different questions. Average quality summarizes the batch. Estimated loss sums the mass of cohorts whose modeled quality falls below the declared threshold of 60. Do not calculate estimated loss as shipment quantity multiplied by one minus average quality.

The reproducible original example

Original unchanged FR-101 seed	Calculated result
Produce and mass	Tomatoes, 500 kg
Entered delay	3 hours
Continue as planned	Quality 58.3 out of 100; 350 kg below threshold; infeasible
Seeded top feasible option	Eastside receiving center
Winner quality and cost	Quality 74.0; modeled loss 0 kg; declared extra cost INR 450

> *Say to faculty: These numbers are reproducible model outputs for an unchanged fictional fixture. They are not measured rescued food, guaranteed financial savings or food-safety certification.*

Evidence: docs/phase-3/seeded-incident.json and src/model/engine.ts. If the live record has changed, describe its current result instead of insisting that it must match this table.

7 Save the analysis and explain History

Create the record that later actions refer to

1. In the editable Rescue Lab, enter an Analysis name that identifies the scenario.
2. In AWS mode choose Run cloud analysis. In local mode choose Save analysis. These are different operations and labels.
3. For the cloud job, open Workflow hub to inspect the queued/running/completed trail. Wait for a successful saved analysis rather than treating submission as completion.
4. Open History and reopen the saved result. Show its input assumptions, parameter/model version and result. The saved view is historical evidence.

Why save before confirming

An analysis is a frozen record of what the system calculated from particular sources at a particular time. A reservation refers back to that record. This makes it possible to answer which assumptions justified the decision. Merely leaving an unsaved calculation on screen does not provide the same durable record.

What happens in AWS

The API validates the request and creates an analysis job. Step Functions coordinates validation and calculation. The Analysis Worker calculates on the server and persists the result with job state. DynamoDB stores the saved run and its owner context. The job trail lets the UI show a genuine asynchronous operation.

Explain a stale result

If a shipment or receiving rule changed after the analysis, the historical record stays visible. The app can block confirmation based on stale sources. Recalculate using current data. Do not edit the historical result to make it look current.

> *Say to faculty: History gives the project an audit trail at the business level. I can reopen exactly what was calculated, even if the source shipment changes later. CloudTrail is a different audit layer: it records AWS API activity.*

8 Use the Rescue copilot and review a draft

Where to find it

Open a saved analysis and find Decision studio. Its tools include Rescue copilot, Delay sensitivity and Split allocation. Live AI requires the AWS workspace. An editable local preview alone is not a saved cloud run for the model to explain.

1. In Rescue copilot, ask a focused question such as Explain the receiving constraints or Summarize a handover brief.
2. Choose Explain result. Read the response together with its source/fallback and cached information. Do not claim a new model call if it is cached.
3. To explore a future case, ask What if the delay is 2 hours and budget is INR 900, then choose Draft scenario.
4. Review the proposed fields and units. Choose Open new scenario only if the draft makes sense. This opens editable assumptions; save/run the new analysis separately.
5. Download brief if you want a text handover. It is distinct from the reservation's private PDF report.

Why the copilot is limited

The deterministic engine supplies authoritative facts. The server accepts only a structured response that points to permitted facts and validated draft fields. The copilot can help explain a decision but cannot certify food safety, invent accepted facts or reserve capacity. Its write permissions are restricted to AI cache/quota state.

What to say to faculty

> *Say to faculty: AI improves the explanation and helps create a what-if draft. The calculation and transaction remain application-controlled. I review the draft before opening a new scenario, and confirming a reservation remains a separate explicit action.*

The business interface uses neutral AI wording. If asked about implementation, answer accurately: the current adapter uses the Groq API with openai/gpt-oss-20b. Bedrock is an optional adapter, not a successfully used provider in the recorded deployment.

9 Show sensitivity and split allocation

Delay sensitivity

Select Delay sensitivity in Decision studio. This explores how results change when the future delay assumption changes. It is a deterministic scenario analysis of the saved run, not a weather forecast or model prediction of traffic. Use it to explain why the decision may become more urgent as an entered delay grows.

1. Start from the saved analysis. Point to the base assumptions.
2. Read the sensitivity rows and compare the quality/loss or feasibility response as delay changes.
3. Explain that the saved base record is not overwritten by this exploration. If you decide to use different assumptions, create and save a new analysis.

Split allocation

Select Split allocation. This considers dividing a shipment across receiving destinations when one receiver may not be sufficient. The calculation conserves the shipment mass, avoids duplicate destination allocations and checks aggregate budget and capacity. The displayed suggestion is not itself an approved cloud booking.

1. Inspect each proposed leg, destination quantity, cost and the total.
2. Explain why total allocated mass must equal the shipment quantity and why capacity must be checked per receiver.
3. In cloud mode, review the allocation before explicitly reserving it with the available confirmation control. If it is infeasible or stale, recalculate instead of forcing it.

The difference to emphasize

> *Say to faculty: A what-if result explores a possibility. A split suggestion proposes how to divide the batch. A confirmed cloud allocation is a separate transaction that checks current capacity. These three stages should not be treated as the same thing.*

If no split can be suggested under the current assumptions, explain that result. Do not present an empty or rejected allocation as a successful feature execution.

10 Review an option and reserve capacity

The approval step

1. In a saved analysis choose a feasible option and select Review cloud plan. In local mode the label is Review local plan.
2. Read the candidate and Snapshot summary. Check destination, quantity, arrival assumptions, quality, cost and source currency.
3. In the AWS workspace, choose Reserve receiving capacity only after explaining that you are approving an operational state change.
4. In local mode, Save local plan records a browser-local plan. It is not an atomic shared-capacity reservation.
5. Open Fulfilment to see the confirmed cloud reservation and its connection to the saved analysis.

Why a second check is necessary

Capacity can change after a calculation was saved. Another reservation may consume the available space, or a receiving rule may have a newer version. The server checks the current state when confirming. A rejected confirmation is evidence of protection against stale assumptions, not automatically a bug.

How double booking is prevented

The implementation uses conditional/transactional DynamoDB writes and stable operation identities. Quantities are represented as whole grams and money as integer paise. A shipment can have one active allocation. Duplicate retries must not subtract capacity twice. A saved score from the browser is not trusted as authority for a booking.

> *Say to faculty: This is the point where a simulation becomes an approved plan. The server checks current versions and capacity atomically, which means the related state changes succeed together or are rejected together.*

For a timed faculty demonstration, prefer showing an existing fictional reservation. Only create a new one when you intend to consume that demo capacity; do not repeatedly click confirmation just to create visible activity.

11 Follow the fulfilment lifecycle

What to inspect

1. Open Fulfilment from the sidebar. Use the status filter to find the relevant reservation.
2. Read its quantity, destination/allocation legs, saved analysis link and state history.
3. On a confirmed fictional reservation, Mark dispatched moves it to DISPATCHED. Do this only when that is the intended demonstration state.
4. On a dispatched reservation, Record delivery moves it to DELIVERED. This records the operator's statement; it is not automatic GPS proof.
5. Cancel is available while CONFIRMED. Explain what cancellation means before using it.

Current state	Allowed next state	Capacity effect
CONFIRMED	DISPATCHED or CANCELLED	Held until an allowed terminal transition
DISPATCHED	DELIVERED	Held until delivery is recorded
DELIVERED	No next state	Released once
CANCELLED	No next state	Released once

Why this is more than a status badge

Each transition is validated on the server. A delivered or cancelled record cannot be reactivated through an arbitrary UI change. Repeated requests for the same state are handled without releasing capacity twice. This is a small business state machine distinct from the Step Functions analysis workflow.

> *Say to faculty: The status buttons represent allowed business transitions. Capacity is released once on delivery or cancellation. Recording delivery shows what an operator entered; it does not prove the physical shipment arrived without an external logistics integration.*

Use View analysis to connect the operational record back to its frozen calculation. That traceability is a useful final demonstration of the full flow.

12 Open reports and explain notifications and outcomes

Private report

In Fulfilment choose Private PDF for a reservation. Report creation happens asynchronously after the business event, so a newly confirmed item may need a short wait and refresh. The authorized API checks ownership and provides a short-lived download link to an object in the private reports bucket. A filename or guessed object key alone should not grant access.

Notifications

Inspect the in-app notification record and its read/acknowledge control. This is stored application state. SNS is a separate publish/subscribe channel. Creating an inbox record, successfully publishing to SNS and a person receiving an email are three different observations. The latest recorded AWS check still shows the rescue email subscription as PendingConfirmation; do not promise receipt during the demo.

Outcome analytics

1. Open Outcome analytics. In local mode, explain that sign-in is required and local plans do not count as completed deliveries.
2. In cloud mode, show delivered quantity, delivery count, active/cancelled work, extra cost and produce breakdown where records exist.
3. Explain the estimate labels. The model-estimated loss remains a model output even after a delivery is recorded.
4. If there are no delivered records, describe the empty state and required workflow rather than inventing completed work.

The summary uses delivered reservations, selecting the latest delivered entry per shipment to avoid counting that shipment repeatedly. Delivered kilograms are operator-recorded fulfilment quantity. They should not be relabelled as independently measured waste prevented.

Data exports

Use Export outcome data on the analytics screen to retain the available outcome data. In Cloud data, Export current snapshot downloads the signed-in workspace snapshot and Reload from AWS reloads it. A snapshot export is not a complete disaster-recovery backup of every bucket, queue and service. Use Fulfilment reservation state, rather than a legacy saved-plan count, as the operational capacity record.

> *Say to faculty: This completes the journey from a calculated option to a recorded outcome. The private report preserves the decision record, while analytics separates operator-recorded fulfilment from illustrative model estimates.*

13 Manage receivers and import shipment data

Receiving network

Open Receiving network. Inspect accepted produce, capacity and receiving hours. These are business constraints used by the model and reservation logic. Cloud administration is restricted to the appropriate authority; an ordinary operator should not silently expand capacity. Editing a receiver changes the source assumptions, so calculate a new analysis before confirming an older plan.

Workflow hub and CSV import

1. In the AWS workspace open Workflow hub. Download the available CSV template instead of guessing its headers.
2. Use fictional rows. The current upload limit is a non-empty CSV no larger than 128 KB. Follow the template's history notation such as 2@10;1@18 for duration/temperature segments.
3. Select the file and start the import. The API prepares a private, scoped upload and creates a job. The worker validates the CSV before committing records.
4. Watch Your workflow trail. It refreshes while the tab is visible. Submission, upload and job success are separate stages.
5. For a failed job, read the validation errors and correct the file. For success, open the saved shipment records. Do not assume a loading indicator means success.

Behind the scenes

API Gateway and the workspace API validate the request, the S3 data bucket receives the scoped upload, SQS buffers work and the Import Worker validates/commits shipment data. Activity stores job history. This separates file handling from the interactive browser and allows failures to be diagnosed.

> *Say to faculty: CSV import demonstrates a real asynchronous cloud workflow. The browser can submit work and show progress while a queue worker validates it. A failed input is reported instead of being presented as a successful shipment import.*

Local data export and reset controls are separate browser tools. Never use a reset as part of a faculty walkthrough unless you intentionally want to clear that fictional browser workspace.

14 Present the evidence and use presentation mode

Evidence screen

Open Cloud and project evidence in the sidebar. Explain the architecture, filter the evidence cards, expand expected/observed results and show cost/lifecycle and study-library sections. The screen is a dated evidence collection, not a live health dashboard. An old passed experiment does not prove that its deleted resources are currently running.

Seven minute presentation guide

1. Select Seven-minute presentation in the sidebar. The panel gives seven segments, notes, navigation and a timer.
2. Use Open screen or the segment arrows to navigate. Start/pause the timer when appropriate. Hide notes when they obstruct content and close the guide when finished.
3. Follow incident, calculation, saved evidence, AI, fulfilment, architecture and limitations. The segment timings total seven minutes.
4. Remember that guide navigation does not calculate, save, reserve, dispatch or deliver anything automatically. You still perform and explain those actions intentionally.

Show a strong piece of proof

A useful evidence example is EC2 replacement: one worker was terminated, the Auto Scaling group created a new healthy worker and the shared EFS file checksum was still available. Another is DMS: full load reached 100 percent with one table, zero errors and three exact matching rows. Then state that those lab resources were deleted.

> *Say to faculty: This screen connects the working product to the syllabus. I separate services still deployed, experiments that were verified and removed, and concepts that are studies. The JSON records support particular observations rather than a general claim that every service is running.*

The latest handover records one real import DLQ alarm. Do not hide the finding or describe all monitoring as green. Explain how an operator would inspect and reconcile failed work before replay.

15 Read the architecture as a sequence of requests

When someone opens the website

The browser requests the HTTPS website. The deployed frontend uses API Gateway and a Web Lambda that reads the private web S3 bucket. The returned HTML loads the React interface. CloudFront was intended as an edge-delivery option, but account verification blocked its creation, so this approved HTTPS fallback is the actual architecture.

When an operator reads or changes a shipment

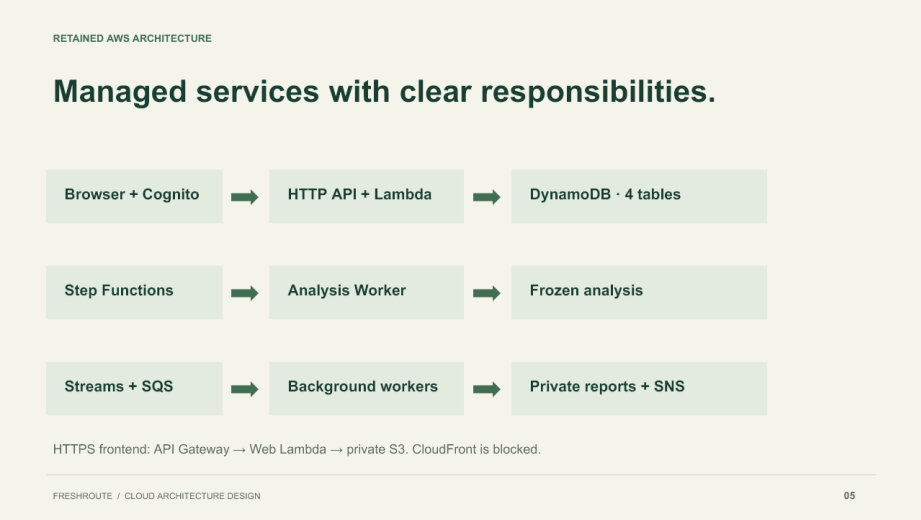
Cognito handles cloud sign-in. The browser sends an access token to the application HTTP API. API Gateway and backend checks establish identity and authorization. Workspace API Lambda validates the command and accesses the owner-scoped DynamoDB records. A response updates the UI. The browser does not receive the server's AWS credentials.

When an analysis runs

The API creates a job and starts Step Functions. Validation runs before CalculateAndPersist. The Analysis Worker calculates the authoritative result and records state. The browser reads the job trail and opens the saved analysis when successful. A preview calculated in the browser is useful for exploration but is not trusted as the authoritative saved cloud result.

When a reservation creates follow-up work

A successful transaction records the decision/capacity change. DynamoDB Streams exposes the event; Outbox Bridge sends post-decision work to SQS. Post-decision Worker creates the private PDF and notification state, and publishes the notification channel. CloudWatch observes execution; CloudTrail records selected AWS activity. The AI explanation is an optional branch from a saved analysis, not the authority that confirms capacity.



Simplified retained application flow from the editable project presentation.

Read the diagram in groups. The full editable flow is in docs/phase-9/ARCHITECTURE.md; its labels distinguish retained services from deleted labs.

16 Amazon S3 and the four bucket design

What the service is

Amazon S3 stores objects: file bytes plus metadata under keys. It is useful for release assets, uploads, reports and audit files. It is different from an attached disk or a shared mounted filesystem. [R2]

Bucket purpose	How FreshRoute uses it	Why separated
Web	HTML, hashed JavaScript/CSS and interface assets	Deployment writers and web reader have a specific boundary.
Data	Private CSV uploads and project fixtures/artifacts	Uploads need scoped write/read rules and temporary-data retention.
Reports	Generated owner-bound private PDF files	Only authorized report workflows should write or issue download links.
Audit	CloudTrail logs and digest files	Audit delivery and retention should be separate from routine application files.

Protection and lifecycle

The four project buckets are private, versioned and require TLS. Reports and audit use the customer-managed KMS key. Versioning supports retaining earlier object versions; a coordinated frontend release must keep the matching entry page, config and assets. Lifecycle settings handle appropriate expiry/noncurrent versions and incomplete uploads. A configured rule is not evidence that an object has already moved to an archive class.

Explain a private download

The API first checks who is requesting the reservation report. It then issues a short-lived signed URL. The URL grants bounded access to that particular object, so it should not be published in a report or treated as a permanent public link. Direct anonymous access to private S3 was tested as denied.

> *Say to faculty: I used four buckets because the access, writers and retention needs are different. I did not create extra buckets per fruit type simply to increase service count. S3 stores files; DynamoDB stores structured application records.*

Source: infrastructure/app.ts; report workflow code; Phase 9 and 10 release-byte/private-S3 checks.

17 DynamoDB and DynamoDB Streams

DynamoDB is retained application storage

DynamoDB is a managed key-value/document database. FreshRoute uses four tables: Shipments, Runs, Destinations and Activity. These support record lookups and histories rather than arbitrary relational joins. On-demand capacity suits a small, variable academic workload. Point-in-time recovery and retention protections are configured for important data. [R3]

Table	Project responsibility
Shipments	Entered shipment data and versions
Runs	Frozen analysis inputs, model version and results
Destinations	Receiving rules and shared capacity state
Activity	Jobs, notification/business activity and AI cache/quota state

Owner-aware keys and backend ownership checks prevent a signed-in user from reading another user's private records merely by guessing an ID. The JobsByOwnerTime index supports a specific history query. An index is created for an access pattern, not because every table should have many indexes.

Transactions and expiration

Conditional writes reject conflicting versions. Reservation transactions coordinate related record/capacity changes. The business objective is that two operators cannot both silently take the same last capacity. Some cache/quota records use TTL expiration; TTL is housekeeping and should not be described as an instantaneous deletion guarantee.

Streams connect a data change to background work

DynamoDB Streams exposes table changes to the Outbox Bridge. The bridge uses the decision event to enqueue post-decision work. A stream is not a replacement for the table, and it is not an exactly-once business guarantee. Downstream workers must tolerate duplicate delivery without generating duplicate bookings.

> *Say to faculty: DynamoDB stores the state; Streams tells another part of the application that relevant state changed. Conditional transactions protect business correctness, and idempotent consumers make event retries safe.*

Source: infrastructure/app.ts, server/store.ts, server/jobs.ts, reservation/outbox handlers.

18 AWS Lambda and the eight function responsibilities

What Lambda contributes

Lambda runs application handlers without this project maintaining an always-running application virtual machine. A request, queue batch, stream event or workflow task can invoke a handler. This reduces idle server management for a variable demonstration workload, but does not remove code, permissions, logs, quotas or failure handling. [R1]

Function	What it does in FreshRoute
Web	Reads private web assets and serves the HTTPS frontend fallback.
Workspace API	Validates authenticated operations and manages application records.
Verification	Runs controlled deployed checks; not the business decision engine.
Import Worker	Validates queued CSV work and commits accepted shipment records.
Analysis Worker	Validates/calculates/persists analysis work coordinated by Step Functions.
Outbox Bridge	Converts relevant stream changes into queued follow-up work.
Post-decision Worker	Generates reports, notification records and publish actions.
Copilot	Explains saved facts or proposes validated future assumptions.

Why separate responsibilities

A slow file import should not hold the user's interactive request open. A report failure should not require the operator to create a second reservation. Separate functions also allow narrower permissions: the Web reader does not need to reserve capacity, and the Copilot cannot write the general workspace.

> *Say to faculty: I use Lambda for distinct responsibilities, not eight unrelated mini-projects. API requests, queue workers, stream events and workflow tasks are different execution triggers in one coherent application.*

Source: infrastructure/app.ts, workflows.ts, fulfilment.ts and copilot.ts. Timeout or retry does not mean work is guaranteed to complete, so the UI and job records still expose failure states.

19 API Gateway Cognito and IAM

API Gateway is the controlled front door

The application HTTP API routes browser requests to backend handlers. The fallback frontend has its own HTTPS API Gateway/Web Lambda path. Distinguish these two uses: serving the interface is different from authorizing a request to change a shipment. The project uses the default HTTPS endpoints; no custom DNS or certificate service deployment is claimed. [R4]

Cognito identifies the application user

Cognito provides the sign-in flow. The browser uses PKCE, which supports a public browser client without embedding a client secret. The returned access token is sent with API calls. The project's session lasts 15 minutes and no refresh token is stored by this app. The operator enters credentials at Cognito, not into a hard-coded demo password field. [R6]

IAM controls AWS service permissions

IAM roles and policies determine what a backend component can do in AWS. Roles are used for application execution and service integration. The project does not require sharing an administrator access key with the browser. Human IAM users/groups and MFA are discussed as access-design topics; they are distinct from Cognito application users.

Authentication is not enough

Authentication answers who is signed in. Authorization answers which record/action that identity may access. A valid token for user A does not authorize user A to read user B's saved run. Both backend owner checks and scoped IAM controls matter. Administrator-only receiving-rule changes add another business authorization boundary.

> *Say to faculty: API Gateway receives the request, Cognito establishes user identity, the application checks record ownership and IAM limits the backend's AWS permissions. These are related but different layers.*

Proof comes from unauthenticated/forged-token rejection, cross-owner access denial and actual owner-scoped requests in the regression evidence, not from merely showing a sign-in screen.

20 Step Functions SQS and dead letter queues

Step Functions coordinates the analysis

The retained Standard workflow makes validation and calculation stages visible. The implemented path has ValidateAnalysisInputs followed by CalculateAndPersist, with failure recording and configured retry/catch behavior. It does not have a separately deployed Lambda for every descriptive phrase such as rank or compare; those calculations occur inside the worker. [R5]

SQS buffers work

The import queue and post-decision queue separate two different workloads. Imports may fail because input is invalid; report/notification work has a different trigger and recovery path. Queues let a browser request return without waiting for all background work, and workers can process available messages under bounded settings. The queues are not the permanent database of business decisions.

What a DLQ means

After configured repeated processing failures, a message can move to a dead-letter queue for investigation. The project has import and post-decision DLQs plus an outbox stream failure queue. Retry is not magic: the cause may still exist, and duplicate delivery must not produce duplicate side effects. Idempotency means repeating the same operation does not incorrectly repeat its business effect.

Use the real operational finding

The Phase 10 snapshot shows three visible messages in FreshRouteImportDLQ and an active depth alarm. They were preserved; no message was purged or replayed to obtain a green status. An isolated Phase 8 experiment separately demonstrated a poison message reaching a lab DLQ and a corrected replay. Do not confuse that isolated success with clearing this production queue.

> *Say to faculty: Step Functions organizes stages; SQS buffers work; a DLQ preserves failed work for diagnosis. I must inspect and reconcile a failed job before replaying it, because retries can repeat side effects unless the application protects them.*

Source: infrastructure/workflows.ts; server/importWorker.ts; outbox/post-decision handlers; Phase 8 reliability and Phase 10 operations evidence.

21 SNS KMS and Secrets Manager

SNS delivers published notifications to subscribers

The rescue topic is used by approved decision-related notification work. Operational alarm notifications use a separate topic. SNS is useful for publishing an event to subscribers, while SQS is useful for buffering jobs for workers. A successful publish and an in-app inbox entry do not prove that someone received an email. The latest recorded rescue subscription is PendingConfirmation. [R5]

> *Say to faculty: The application's notification state is visible in the app. Email is a separate delivery channel and remains unverified until the subscription and a controlled receipt test succeed.*

KMS manages encryption keys

The project uses a customer-managed key for report and audit object encryption, with rotation and scoped access policies. This adds key-permission controls beyond merely hiding a URL. The application must have the right object and key permissions for an authorized operation. Encryption at rest does not replace HTTPS in transit or owner authorization. [R6]

> *Say to faculty: S3 stores the encrypted report object; KMS controls the key used for the encryption integration. The user still needs application authorization before receiving a temporary report link.*

Secrets Manager stores the AI credential

The server-side API credential is configured as freshroute/copilot-api-key under the JSON field apiKey. CloudFormation resolves the configured secret reference into the server runtime. It is not in cloud-config.json, the browser bundle, the student report or slides. Never open the secret value to demonstrate that it exists. Show metadata/configuration without revealing the credential.

The reason for this service is to separate confidential configuration from application source. It does not mean the browser calls the model directly. The Copilot handler makes the server-side provider request, with token/deadline/quota boundaries and deterministic fallback.

Source: infrastructure/app.ts, infrastructure/copilot.ts and server/copilot.ts. [R6]

22 Explain the AI integration in technical detail

The request and response path

A user opens a saved run and sends a question to the authenticated copilot endpoint. The server checks access to that run and constructs an authoritative fact set. It applies cache/quota rules, calls the configured provider adapter when required, validates the structured output, then returns an explanation or a deterministic fallback. It does not accept arbitrary model text as a business command.

Why facts are separate from generated wording

The calculation engine supplies option IDs, constraint facts and numeric results. The model selects permitted facts and provides explanatory wording. Draft fields are validated before the user can open an editable scenario. A valid-looking JSON response is insufficient if it points to unsupported facts or exceeds allowed fields. This helps prevent a persuasive answer from silently changing the decision.

The bounded configuration

Control	Purpose
800 output tokens	Bound response size and model usage.
Nine second deadline	Do not leave the user waiting indefinitely.
One hour cache	Reuse a matching owner/run/question/model response.
20 owner and 200 global daily uncached attempts	Limit usage; failed uncached attempts count.
AI-only write scope	Keep model orchestration away from reservations/capacity.
Visible fallback and provenance	Make the source of an explanation clear.

Answer the provider question accurately

The deployed technical path recorded in the project uses Groq, model openai/gpt-oss-20b. A Bedrock adapter exists as an optional implementation path; account restrictions prevented a successful Bedrock-use claim. Neutral AI labels in the business UI are appropriate, but saying Bedrock was used when it was not would be inaccurate.

> *Say to faculty: AI is an explanation and drafting layer over a reproducible engine. A suggested scenario must be reviewed and calculated again. The model has no authority to reserve capacity or certify food safety.*

23 CloudWatch CloudTrail and AWS Config

CloudWatch observes operation

CloudWatch collects logs/metrics and supports dashboards and alarms. FreshRoute retains a dashboard and 14 alarms for relevant errors, workflows, queue depths and other operational signals. A canary test observed OK to ALARM and back to OK with successful SNS actions. The Phase 10 snapshot has 13 OK and one ALARM; some OK error alarms treat missing data as non-breaching, so OK alone is not proof of recent successful traffic. [R8]

CloudTrail records AWS activity

CloudTrail provides an audit trail of AWS API actions. The project observed actual encrypted audit objects and digests in the audit bucket and selected a real Auto Scaling termination event. Management writes and scoped report-object data events are configured separately. Digest delivery and enabled validation were observed; full cryptographic digest-chain validation was not performed. [R9]

Config evaluates resource configuration

The disposable Config experiment evaluated a narrow security-group condition. It observed COMPLIANT, deliberate temporary drift to NON_COMPLIANT and COMPLIANT after remediation. The temporary recorder/rule and lab were removed afterward. This is proof of that configuration experiment, not a retained continuous compliance certification. [R10]

Question	Service to discuss
Is a worker producing errors or is the queue growing?	CloudWatch
Which AWS API action changed something?	CloudTrail
Does a recorded resource configuration satisfy a rule?	AWS Config

> *Say to faculty: These three services answer different questions: operational health, API activity and configuration compliance. The application's own shipment history is another layer and should not be confused with any of them.*

Evidence: docs/phase-8/operations-final-verification.json and config-verification.json; docs/phase-10/CLOUD_CHECKS.json.

24 CloudFormation CDK and Systems Manager

CloudFormation manages related resources as stacks

The retained deployment has Foundation, Application and Operations stack boundaries. Templates describe resources and dependencies rather than requiring a person to remember every console click. A completed stack status means the deployment operation completed; it does not prove that every business flow, email or recovery scenario works.

CDK produces infrastructure definitions from code

FreshRoute uses TypeScript CDK for the main Foundation/Application architecture, with operational/lab templates and scripts for the additional experiments. Synthesis generates the templates for review. It is not the same as deploying them. Source parameters control provider/configuration choices, while secret references keep credential values out of source. CloudFormation is the deployment mechanism, not another website end-user feature.

Why this matters: the deployment can be reviewed, reproduced and repaired. The source also records deliberate retention protections for important data. A stack delete does not necessarily delete retained buckets, versions, tables or keys. Reproduction in a different AWS account requires that account's correct identity endpoints and parameters.

Systems Manager operates the EC2 lab

Systems Manager provided managed Run Command access to the temporary workers. The lab used this channel to inspect file content and connectivity without exposing SSH as the routine operations path. A managed session still requires permissions, a healthy agent and a valid network path to service endpoints; it is not a reason to omit network design. The workers had temporary controlled egress for required setup/SSM access.

> *Say to faculty: CDK defines infrastructure in code, CloudFormation creates and tracks the resource stacks, and Systems Manager was used to operate the EC2 experiment. These are engineering and operations tools around the product, not buttons every dispatcher must understand.*

Source: infrastructure/app.ts and infrastructure/labs; project deployment notes and compute verification. Generic service definitions: [R12, R13].

25 Budgets cost review Free Tier and Trusted Advisor

Budgets is a warning mechanism

The project planning ceiling is USD 200. Its configured gross-cost thresholds are USD 25, 50, 100, 150 and 180. The goal is to notice spend before it becomes uncomfortable. This configuration is not an enforced stop that guarantees charges cannot exceed USD 200. Billing information and notifications are delayed. [R11]

Cost Explorer and billing evidence

Phase 10 records account-wide estimated UnblendedCost excluding credits/refunds, with its UTC query time and covered date interval in OPERATIONS_CHECKS.json. It is gross account activity rather than a final isolated FreshRoute invoice. A tiny returned estimate must not be presented as proof that all the labs were free or that no later charge can appear.

Free Tier and credits

The account review records the plan, remaining credits and expiry as a dated snapshot. Availability depends on the account and time. Credits may offset eligible charges, but the architecture should still be explainable using gross costs. Never assume that every selected AWS service is free because the account has a Free Tier plan.

Trusted Advisor

Trusted Advisor is covered as a recommendations/support topic. The attempted Support API check returned a subscription-required restriction in this account. The project did not buy a higher support plan to claim access. Restricted recommendations remain a study; do not display a successful account-wide optimization audit that was never performed.

What actually reduced ongoing cost

The five engineering lab stacks were deleted and residual resource inventory was checked. The useful main application and operations remain. Removing EC2 alone would not remove an ALB, shared filesystem, database, logs or retained storage, so cleanup had to consider resource families instead of one instance count.

> *Say to faculty: I control cost through bounded work, quotas, retention and deliberate cleanup. Budget alerts help detect spend, while credits and Free Tier are account-specific benefits rather than architecture guarantees.*

Evidence: Phase 8 account/cost/cleanup files and Phase 10 OPERATIONS_CHECKS.json.

26 EC2 Auto Scaling and load balancing lab

EC2 represents the virtual machine alternative

EC2 provides virtual compute instances. The project's temporary lab used two small workers to demonstrate a server-based alternative to the retained Lambda product. Instance type selection was a workload/cost decision, not a claim that a small instance can handle arbitrary production traffic. Instance/pricing model comparisons are syllabus studies; no Reserved Instance or Savings Plan was purchased. [R1]

Auto Scaling maintains a desired group

An Auto Scaling group manages the number and health of workers. The experiment terminated one worker and observed a new healthy replacement. A controlled policy test changed desired capacity from one to two. That proves replacement and this bounded scale-out behavior; it is not a measured high-load benchmark or a multi-region disaster recovery test.

The Application Load Balancer routes requests

The internal ALB checks targets and routes the lab's requests to healthy workers. It was not a public customer-facing endpoint and did not replace the production HTTP API. The lab fixture used internal HTTP; the retained product uses HTTPS. ALB health and ASG replacement complement each other but do not make every downstream dependency highly available. [R4]

What the observation proves

The new worker had a different instance identity and became healthy. It could read the same EFS checksum, while per-instance EBS scratch content was distinct. This connects compute lifecycle to the storage choice: a replaced machine should not be the only location of required shared data.

> *Say to faculty: I verified a small EC2 deployment pattern: health-checked workers, an internal load balancer, replacement and controlled scaling. I removed the lab afterward because the main dispatcher application runs on managed serverless services.*

Evidence: compute-verification.json, replacement-verification.json and scale-and-migration-completion.json under docs/phase-8. All related temporary stacks were deleted.

27 VPC subnets routes security groups and peering

Give each networking concept one job

Concept	Simple meaning	FreshRoute lab use
VPC	An isolated logical network in AWS	Compute lab network and a separate peer network.
Subnet	A network subdivision in an Availability Zone	Two-AZ compute layout and private SQL subnets.
Route table	Rules that choose a traffic path	Explicit routes and return paths for peering.
Security group	Stateful permitted traffic rules around resources	Narrow application, database and managed-operation access.
VPC peering	A private connection between VPC networks	Non-overlapping networks with tested bidirectional connectivity.

Explain allowed and denied traffic

The lab tested both connectivity that should work and SSH access that should not. Creating a peering object alone is insufficient: addresses must not conflict, appropriate routes must exist on both sides and security groups must allow the intended traffic. A subnet is not private merely because of its name; routes and exposure determine the actual path. [R4]

Explain egress honestly

The EC2 setup had a controlled path for packages/Systems Manager. The database subnets had no internet route. Do not draw private instances fetching internet packages without explaining NAT, endpoints or another valid path. These approaches have different costs and must be designed, not assumed.

Boundary of this evidence

VPC peering is not AWS Direct Connect, a VPN or proof that an on-premises warehouse was connected. The retained Lambda product is not claimed to run inside these deleted teaching VPCs. The network lab was removed after capturing evidence.

> *Say to faculty: Routes tell traffic where to go, security groups tell resources what traffic is allowed, and peering connects the two lab networks. I verified the intended path and a denied path, not just the existence of a diagram.*

Evidence: docs/phase-8/compute-verification.json and cleanup-verification.json.

28 EBS EFS and WAF lab

EBS is the worker disk boundary

EBS supplies block storage used by EC2. The lab wrote per-instance scratch data and observed distinct content on the workers. This demonstrates that an attached disk is not automatically a shared filesystem. Persistence depends on the volume configuration and lifecycle; do not claim every scratch file survives instance termination. [R2]

EFS is the shared file boundary

EFS provides a filesystem that the workers can mount under the configured network/access controls. Both workers read the same shared fixture and checksum, including the replacement worker. This is the reason to separate shared data from disposable machine-local scratch. EFS was part of the lab, not a storage dependency of the retained React website. [R2]

Choose this for	Storage model	Project example
An uploaded or generated file	S3 object API	CSV upload, PDF report or audit object
A virtual machine disk	EBS block device	Per-worker scratch content
A mounted shared file	EFS filesystem	Shared lab fixture/template file

WAF filters web requests

The disposable WAF protected the compatible lab ALB. An ordinary request returned 200 while a harmless known test pattern returned 403. This proves the configured request filter, not resistance to every attack. It also does not prove direct WAF protection for the retained API Gateway HTTP API. [R6]

> *Say to faculty: The storage experiment separates object, block and file storage using actual bytes. The WAF experiment separately shows a permitted request and a blocked test request. Both were temporary teaching experiments and their resources are deleted.*

Evidence: docs/phase-8/compute-verification.json and replacement-verification.json.

29 RDS and the DMS migration experiment

RDS provides the SQL teaching environment

The lab used a private PostgreSQL RDS instance with fictional warehouse inventory. SQL suits the legacy/warehouse comparison because it represents relational rows and schemas. It was not a second production source of truth for the main shipment app. A SQL client connection verified TLS 1.3 and certificate checks in the recorded test. [R3]

DMS performs the full load

DMS copied a fictional table from a legacy schema to a warehouse schema on that same disposable RDS instance. The final task reached 100 percent, one table loaded and zero tables errored. Source and target had three exact matching rows. This is a bounded full-load demonstration with reconciliation, not an on-premises migration or a production cutover. [R7]

Why reconciliation matters

A green task status alone does not prove correct target content. The experiment compared row counts and actual values. There could otherwise be missing or altered rows despite a task finishing. The client's verified SQL TLS connection and the DMS endpoints' require-encryption setting have different certificate-verification guarantees; do not describe them as identical.

What was not tested

The lab did not demonstrate change data capture, long-running replication, Multi-AZ database failover, read scaling or a real production cutover/rollback. DMS source/target schemas were on one instance. The source, target and migration components were deliberately disposable, and the lab stacks were deleted after verification.

> *Say to faculty: I used a tiny relational migration to demonstrate the migration process and content validation. I can show completion and exact reconciliation for three fictional rows. I do not claim CDC, external warehouse migration or production cutover.*

Evidence: docs/phase-8/migration-verification.json, scale-and-migration-completion.json and cleanup-verification.json.

30 Explain services that remain studies

CloudFront and Direct Connect

CloudFront is an edge content-delivery service that can cache web content near users. Its creation is blocked by account verification here; API Gateway/Web Lambda is the deployed HTTPS fallback. Direct Connect represents a dedicated private connectivity option for a warehouse/enterprise network. FreshRoute did not provision a physical circuit. Explain the future fit without saying either service is active. [R4]

RDS Multi AZ read replicas and Aurora

A Multi-AZ design targets database availability/failover. A read replica targets read workloads and brings replication-lag/routing considerations. Aurora is a relational cluster alternative. These address different requirements; one should not be presented as an interchangeable label for another. The actual SQL fixture was Single-AZ, and these alternatives were not deployed/tested. [R3]

S3 classes and Glacier

Hot application objects fit readily accessible storage. Archival classes trade access behavior and retrieval/retention costs for colder storage needs. The project explains these choices and configures lifecycle policies where appropriate, but it did not prove a real Glacier archive transition or restore. A tiny configuration object is not a sensible prop for claiming every storage class. [R2]

Shield and SWF

Shield is the syllabus DDoS-protection comparison; no paid Advanced subscription or DDoS experiment is claimed. WAF application-request filtering is not the same thing as proving DDoS resilience. SWF is a workflow/worker-coordination study compared with the chosen Step Functions design; no SWF workflow is deployed. [R6, R5]

Historical migration and physical transfer topics

SMS is discussed as the historical server-migration syllabus concept. Snowball represents offline data transfer using a device; Snowball Edge adds an edge-processing/transfer scenario; Snowmobile represents the historical very-large-scale physical transfer concept. None was ordered or provisioned. Product availability changes over time, so this guide explains their syllabus role without asserting current orderability. [R7]

> *Say to faculty: I covered these topics through a design comparison because deploying expensive, physical, restricted or unsuitable services would not improve this student application. The syllabus matrix labels that boundary explicitly.*

31 Explain security reliability and architecture choices

Shared responsibility

AWS operates the underlying cloud infrastructure for managed services, while the project still owns data classification, application validation, access rules, secrets, workload configuration and safe operations. The responsibility changes with the service: an EC2 operator has more machine/OS work than a Lambda operator. Using AWS does not automatically make an application compliant.

Security story in one chain

HTTPS protects the connection; Cognito identifies the user; API/backend checks restrict actions; owner keys and IAM policies narrow access; private S3 and KMS protect artifacts; Secrets Manager separates confidential configuration; CloudTrail and CloudWatch support audit and operations. Each control has a different job and none substitutes for the others.

Reliability story

Jobs and queues make background progress observable. Retry/catch paths and DLQs expose failure. Idempotent operations and conditional writes protect state during retries. The ASG lab observed machine replacement. These are concrete mechanisms, but no uptime SLA, complete disaster recovery validation or multi-region failover is claimed.

The six Well Architected pillars

Pillar	FreshRoute example
Operational excellence	Versioned infrastructure, job trails, runbooks and dated evidence.
Security	Private artifacts, owner checks, bounded permissions and secret separation.
Reliability	Conditional state changes, retries, DLQs and observed replacement.
Performance efficiency	On-demand managed services and separated background workloads.
Cost optimization	Deleted labs, quotas, lifecycle rules and gross-cost alerts.
Sustainability	Avoid retaining idle teaching compute merely for service count.

Say to faculty: The cloud depth is in the interactions, permissions, state transitions, failure handling and evidence. Counting service logos alone would not demonstrate architecture quality.

32 Seven minute faculty demonstration script

Use this as a speaking route, not as a command to change real operational records. Rehearse on fictional data. The times total 420 seconds. If you show the full slide deck instead, use its separate seven-minute timing rather than trying to present both inside one slot.

Time	Screen	Suggested words
0:00 to 0:45	Shipment detail	This fictional shipment is delayed. The entered quantity, temperatures and journey assumptions are the evidence for the decision. FreshRoute helps a dispatcher compare practical rescue options.
0:45 to 2:10	Rescue Lab	I change one assumption, then compare continuation, cooling and rerouting. Hard constraints remove invalid options before ranking. These are illustrative model outputs, not a food-safety certificate.
2:10 to 3:10	Saved analysis and History	I save a cloud analysis and reopen the frozen result. Its model and source versions preserve the reason for the decision. Changed source data requires recalculation before confirmation.
3:10 to 4:10	Decision studio	The copilot explains saved facts and can draft a what-if case. I review the fields before opening a new scenario. AI cannot reserve capacity, and a fallback is labelled.
4:10 to 5:20	Fulfilment	This existing fictional reservation checked current capacity. The state machine controls dispatch, delivery and one-time release. The private report requires authorization.
5:20 to 6:30	Cloud evidence	The retained app uses managed APIs, identity, compute, storage, queues and workflows. The EC2/network/SQL experiments have measured evidence and were deleted. Studies are labelled.
6:30 to 7:00	Evidence or summary	The value is a reproducible path from incident to reviewed action. The model is illustrative, email receipt is unverified, CloudFront has an account gate and budget alerts are not hard spending caps.

If faculty interrupts, answer with the relevant service’s role and point back to the user action. For example: SQS appears because report/import work should not block the interactive dispatcher request.

33 Questions faculty may ask and clear answers

Why not do everything in one Lambda

One handler could be simpler initially, but imports, interactive commands and post-decision reports have different timing, permissions and failure modes. Separating them prevents a report retry from requiring another reservation and permits a narrower AI role.

Why both DynamoDB and RDS

DynamoDB is the retained product database chosen for owner/ID lookups, histories and conditional capacity transactions. RDS represented the relational legacy warehouse in a temporary teaching lab. They are not two conflicting active sources of truth.

How do you know the AI answer is correct

I do not assume all generated wording is correct. The deterministic engine owns the numbers, the server validates structured references and draft fields, and the operator reviews the explanation. Unsupported or unavailable output falls back. This reduces risk without proving the model can never be wrong.

Is every syllabus service deployed

No. The 71-topic matrix covers all eight modules with retained implementation, verified/deleted experiment, blocked/restricted status or study. I can explain every service’s purpose without falsely claiming a physical circuit, archive restore or obsolete migration deployment.

What does zero estimated loss mean

It means no modeled cohort is below the declared threshold for that scenario. It does not mean no produce can spoil, that food is safe or that loss was independently measured.

What would make this production ready

Calibrated quality models, real receiver/dispatch integration, confirmed delivery channels, recovery and load tests, operational ownership and appropriate data protection controls. The existing queue alarm and confirmation issues must also be addressed.

How does this project differ from a static dashboard

It has persistent owner-bound records, authoritative calculations, frozen analyses, asynchronous jobs, validated AI drafts, transactional capacity and allowed fulfilment transitions. Evidence checks actual behavior rather than showing decorative active-service badges.

34 Troubleshooting and honest completion status

Situation	What to do and explain
Cloud sign-in expired	Sign in again through Cognito. Do not paste tokens or passwords into source/config.
No saved analysis for AI	Run/save the analysis first. A preview is not a saved cloud run.
The top option cannot be confirmed	Check current rules, source versions and capacity; recalculate. Do not bypass validation.
AI unavailable or fallback shown	Explain deadline/validation/quota/cache behavior. Use the deterministic result and visible provenance.
New private PDF not ready	Allow background work to finish, refresh and inspect its job/error state; avoid duplicate bookings.
No outcomes shown	Check cloud mode and whether delivery was actually recorded. Do not count a local plan as delivery.
Import DLQ alarm	Inspect/reconcile the owning failed jobs before selective replay. Never purge merely to make the dashboard green.
Email does not arrive	Check current SNS confirmation and controlled delivery independently of app inbox state.
Network unavailable during evaluation	Use the local calculation, saved report/deck and dated cloud evidence. State which parts are offline.

Status at the latest recorded check

Implementation and submission packaging are complete, but operational readiness has open items. Phase 10 observed three visible import DLQ messages, one active alarm, pending rescue email confirmation and completed retained stacks. CloudFront verification remains unresolved. Browser visual/accessibility verification and a human timed rehearsal remain outstanding.

> *Say to faculty: I can show implemented behavior and dated evidence, but I will not call every operational condition resolved. The honest next steps are targeted queue diagnosis, confirmed notification delivery and an allowed-browser rehearsal.*

Do not claim actual food rescued, calibrated safety, measured commercial savings, a new live regression run, full digest-chain validation, database failover or a guaranteed mark.

35 Glossary for a first time reader

Term	Plain meaning
API	A defined way for the browser to request data or an action from the server.
Authentication	Proving who is signed in.
Authorization	Checking what that identity is allowed to access or change.
Serverless	Managed execution without this project operating the application server machine.
Snapshot	A saved copy of inputs/results for a particular calculation.
Feasible option	A candidate that passes the implemented constraints.
Transaction	Related state changes that must succeed or fail together.
Idempotency	A repeated request does not incorrectly repeat its business effect.
Queue	A buffer of work waiting for a consumer.
DLQ	A dead-letter queue that preserves repeatedly failed work for investigation.
Outbox	A pattern that connects a recorded business change to follow-up event processing.
Presigned URL	A temporary signed link authorizing a particular object operation.
Region	A geographic AWS deployment location; this project uses Mumbai.
Availability Zone	An isolated location within a region used in availability designs.
Encryption at rest	Protection of stored bytes using encryption.
TLS	Encryption/protection for a network connection.
Full load	An initial copy of existing source records to a target.
CDC	Change data capture for later data changes; not tested in this project.
Reconciliation	Checking source and target counts and values after transfer.
Gross cost	Cost before the credits/refunds excluded by the recorded query.

Learning these terms is enough to begin the demonstration. Explain the user problem first; introduce each technical word when its role becomes visible.

36 Sources and where to find the proof

Project specific facts

The website instructions were checked against src/App.tsx, Welcome.tsx, CloudRuntime.tsx, RescueWorkspace.tsx, RunReview.tsx, DecisionInsights.tsx, CloudOperations.tsx, Fulfilment.tsx and OutcomeAnalytics.tsx. Model definitions are in src/model/engine.ts, parameters.ts and outcomes.ts. Backend handlers and infrastructure files establish the service responsibilities and access boundaries.

The worked example is docs/phase-3/seeded-incident.json. Deployment and experiment evidence is under docs/phase-4 through phase-8. The 19 business regression checks are Phase 8 evidence. Phase 9 holds the demo script, syllabus matrix and report. Phase 10 has the latest captured health, stack, queue, alarm, subscription, release and cost observations. File paths are relative to the submitted source root.

Official AWS definitions consulted

[R1] Compute overview: https://docs.aws.amazon.com/whitepapers/latest/aws-overview/compute-services.html

[R2] Storage overview: https://docs.aws.amazon.com/whitepapers/latest/aws-overview/storage-services.html

[R3] Database overview: https://docs.aws.amazon.com/whitepapers/latest/aws-overview/database.html

[R4] Networking overview: https://docs.aws.amazon.com/whitepapers/latest/aws-overview/networking-services.html

[R5] Application integration: https://docs.aws.amazon.com/whitepapers/latest/aws-overview/application-integration.html

[R6] Security and identity: https://docs.aws.amazon.com/whitepapers/latest/aws-overview/security-services.html

[R7] Migration overview: https://docs.aws.amazon.com/whitepapers/latest/aws-overview/migration-services.html

[R8] CloudWatch: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html

[R9] CloudTrail: https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-user-guide.html

[R10] Config: https://docs.aws.amazon.com/config/latest/developerguide/WhatIsConfig.html

[R11] Budgets: https://docs.aws.amazon.com/cost-management/latest/userguide/budgets-managing-costs.html

[R12] CloudFormation: https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/Welcome.html

[R13] Systems Manager: https://docs.aws.amazon.com/systems-manager/latest/userguide/what-is-systems-manager.html

Definitions were consulted for this guide on 23 October 2026. They explain AWS capabilities, while the local implementation/evidence establishes what this project actually configured or tested.